

ExoHabitAI

Exoplanet Habitability Prediction Platform

Complete Project Technical Report
Data Preprocessing | Machine Learning | Flask API | React Frontend | Render Deployment

LIVE DEPLOYMENT LINKS

Frontend: <https://exohabitai-2-0eyo.onrender.com>

Backend : <https://exohabitai-1-3xkv.onrender.com>

API Docs : <https://exohabitai-1-3xkv.onrender.com/docs>

(Click links above to open in browser)

Generated: April 04, 2026

Table of Contents

1. Project Overview & Architecture
2. Module 1 — Data Preprocessing
3. Module 2 — Machine Learning Model Training
4. Module 3 — Flask Backend API
5. Module 4 — React Frontend
6. Module 5 — Deployment on Render
7. API Reference — All Endpoints
8. End-to-End Data Flow
9. Challenges & Solutions
10. Project File Structure
11. How to Run Locally
12. Summary & Live Links

1. Project Overview & Architecture

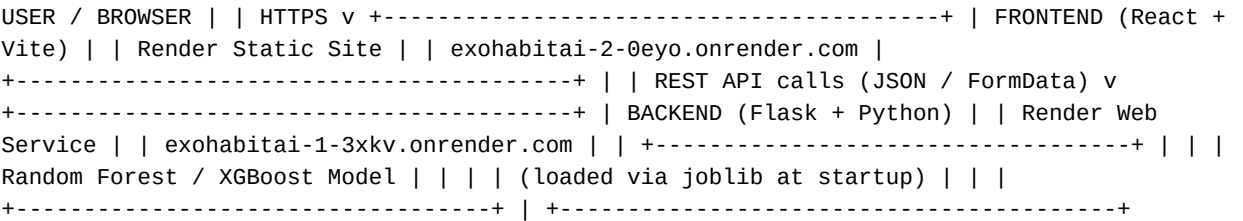
ExoHabitAI is a full-stack AI-powered platform that predicts and ranks the habitability of exoplanets using supervised machine learning. The system accepts planetary and stellar parameters, processes them through a trained ML model, and returns a habitability score along with a human-readable classification. The project was built in five stages: data preprocessing, ML model training, Flask backend API, React frontend, and cloud deployment on Render.

1.1 At a Glance

Project Name	ExoHabitAI — Exoplanet Habitability AI
Domain	Astrobiology / Machine Learning / Full-Stack Web Development
ML Problem Type	Supervised Binary Classification (Habitable vs Non-Habitable)
Primary Model	Random Forest Classifier (with XGBoost comparison)
Backend	Flask (Python) REST API
Frontend	React 18 + Vite + Tailwind CSS
Deployment	Render.com (Free Tier — two separate services)
Live Frontend	https://exohabitai-2-0eyo.onrender.com
Live Backend	https://exohabitai-1-3xkv.onrender.com
API Docs	https://exohabitai-1-3xkv.onrender.com/docs

1.2 System Architecture

The project is split into two independently deployed services that communicate over HTTPS:



2. Module 1 — Data Preprocessing

Before any machine learning can happen, raw exoplanet data from NASA and ESA catalogs must be cleaned, validated, engineered, and standardised. This module covers every step from raw CSV ingestion to producing the final `preprocessed.csv` used for training.

2.1 Expected Features (Input Columns)

The raw dataset must contain the following 10 features:

Feature Name	Description	Unit
pl_rade	Planet Radius	Earth radii
pl_bmasse	Planet Mass	Earth masses
pl_orbper	Orbital Period	Days
pl_orbsmax	Semi-Major Axis	AU (Astronomical Units)
pl_eqt	Equilibrium Temperature	Kelvin
pl_dens	Planet Density	g/cm3
st_teff	Host Star Temperature	Kelvin
st_lum	Star Luminosity	Solar luminosity
st_met	Star Metallicity	[Fe/H]
st_spectype	Star Type	Categorical (G, K, M, F ...)

2.2 Data Quality Assessment

Each dataset undergoes a thorough quality audit to identify:

- Missing values and nulls across all columns
- Duplicate rows (same planet from different catalogue entries)
- Inconsistent units (e.g., km vs Earth radii, Celsius vs Kelvin)
- Physically impossible values (negative radius, temperature below absolute zero)

Summary statistics and a missing-value heatmap are generated at this stage to visually assess data quality before any modification is made.

2.3 Handling Missing Data

Feature Type	Imputation Method	Rationale
Planetary physical values (radius, mass, period)	Mean / Median Imputation	Robust to skewed distributions
Star temperature (st_teff)	Median imputation	Avoids influence of extreme outliers
Categorical — Star type (st_spectype)	Mode (most frequent value)	Preserves dominant class
Rows with >50% missing values	Remove entirely	Too little information to be reliable

2.4 Outlier Detection & Handling

Two statistical methods are applied to detect outliers:

- Z-Score Method — flags values more than 3 standard deviations from the mean.
- IQR (Inter-Quartile Range) — flags values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$.

Actions taken on detected outliers:

- Remove rows with physically impossible values (e.g., negative planet radius, temperature below -273.15 C).
- Cap extreme but plausible values at the 1st / 99th percentile to reduce distortion.

2.5 Unit Standardisation

All features are converted to standard astronomical units for consistency across data sources:

Feature	Converted To
Planet Radius	Earth radii (1 Earth radius = 6,371 km)
Planet Mass	Earth masses (1 Earth mass = 5.97×10^{24} kg)
Orbital Distance	AU — Astronomical Units (1 AU = 149.6 million km)
Temperature	Kelvin (K)

2.6 Feature Engineering

Three new engineered features are derived from the raw columns to provide the ML model with richer, physics-informed signals:

A) Habitability Score Index

A composite score computed from temperature proximity to the habitable range (273–373 K), planet radius similarity to Earth, orbital distance from the star, and stellar luminosity. This converts raw physics into a single ML-readable signal that correlates directly with life-supporting potential.

B) Stellar Compatibility Index

Measures how compatible the host star is for supporting habitable planets. Uses host star temperature, star size (radius), and radiation stability. A high index indicates the star emits stable radiation in suitable wavelengths over geological timescales.

C) Orbital Stability Factor

Derived from orbital period and semi-major axis. Stable, near-circular orbits with moderate periods are associated with better long-term habitability. Highly eccentric or very short/long orbits receive lower stability scores.

2.7 Categorical Encoding

Star type (st_spectype) is a categorical column with classes G, K, M, F, A, and others. One-Hot Encoding is applied to convert each class into a separate binary column, preventing the model from assuming any ordinal relationship between star types.

2.8 Feature Scaling

Numerical features are scaled using `StandardScaler` (zero mean, unit variance) or `MinMaxScaler` (0-1 range) depending on the model. Scaling is applied inside a scikit-learn Pipeline to prevent data leakage between train and test splits.

2.9 Target Variable Creation

The target column `y` is created as a binary variable:

- 1 — Potentially Habitable (planet meets minimum criteria for liquid water)
- 0 — Non-Habitable (extreme conditions incompatible with known life)

An optional multi-class target (High / Medium / Low habitability) may also be created for extended analysis. The final preprocessed dataset is saved as `preprocessed.csv` and committed to `data/preprocessed/` in the GitHub repository.

3. Module 2 — Machine Learning Model Training

This module covers the complete ML pipeline: dataset preparation, baseline modelling, primary model training, hyperparameter tuning, evaluation, model selection, and habitability scoring. The objective is to select the best-performing model for predicting exoplanet habitability potential.

3.1 Problem Formulation

Learning Type	Supervised Machine Learning
Prediction Type	Binary Classification — 1 (Habitable) or 0 (Non-Habitable)
Input	Cleaned and engineered numerical and categorical features
Output	Habitability class label + habitability probability score
Input File	data/processed/exohabit_ml.csv
Train/Test Split	80% training / 20% testing random_state=42

3.2 Baseline Models (Reference Performance)

Before training advanced models, two simple baseline classifiers are fitted to establish a performance benchmark:

- Logistic Regression — linear decision boundary, fast training
- Decision Tree (shallow depth) — interpretable but prone to overfitting

The baseline gives initial accuracy and recall scores. All advanced models must outperform the baseline on F1-score and Recall to be considered.

3.3 Primary Model Candidates

3.3.1 Random Forest Classifier

An ensemble of decision trees that vote on the final prediction. Each tree is trained on a random subset of features and samples (bagging), making the model robust to noise and outliers. Key advantages for this problem:

- Handles non-linear relationships between features and habitability
- Robust to outliers — critical given extreme cosmic measurements
- Provides feature importance rankings for scientific interpretability
- `class_weight='balanced'` corrects for class imbalance in training data

3.3.2 XGBoost Classifier

Gradient Boosted Trees that sequentially correct errors from previous trees. XGBoost is often the top performer on structured/tabular datasets. Key advantages:

- High performance on structured data — often outperforms Random Forest
- Handles complex feature interactions through boosting
- Built-in L1/L2 regularisation prevents overfitting

3.4 scikit-learn Pipeline

A Pipeline chains preprocessing and model steps to prevent data leakage. The scaler is fitted only on training data and applied identically to test data:

```
from sklearn.pipeline import Pipeline from sklearn.preprocessing import StandardScaler from
sklearn.ensemble import RandomForestClassifier pipeline = Pipeline([ ('scaler',
StandardScaler()), ('model', RandomForestClassifier( n_estimators=100,
class_weight='balanced', random_state=42)) ]) pipeline.fit(X_train, y_train)
```

3.5 Model Evaluation Metrics

Every model is evaluated using the following metrics on the held-out test set:

Metric	Purpose	Priority
Accuracy	Overall percentage of correct predictions	Medium
Precision	Of all planets predicted habitable, how many truly are?	High
Recall	Of all truly habitable planets, how many were found?	Critical
F1-Score	Harmonic mean of Precision and Recall — overall balance	Primary
ROC-AUC	Probability-based discrimination between classes	High

Mandatory evaluation outputs:

- Confusion Matrix — shows TP, FP, TN, FN counts
- Classification Report — per-class precision, recall, F1
- ROC Curve with AUC score

3.6 Hyperparameter Tuning

After baseline evaluation, hyperparameter tuning is performed using GridSearchCV or RandomizedSearchCV with 5-fold cross-validation:

```
# Random Forest tuning rf_params = { 'model__n_estimators': [100, 200, 300],
'model__max_depth': [None, 10, 20, 30], 'model__min_samples_split': [2, 5, 10] } # XGBoost
tuning xgb_params = { 'model__learning_rate': [0.01, 0.1, 0.2], 'model__max_depth': [3, 5,
7], 'model__n_estimators': [100, 200, 300] }
```

3.7 Model Comparison & Final Selection

Models are compared on their F1-score and Recall on the test set. Recall is prioritised because missing a truly habitable planet (false negative) is more costly than a false alarm. The winning model must:

- Achieve the best F1-score and Recall on test data
- Show no significant overfitting (train vs test performance gap < 5%)
- Be stable across different random seeds

3.8 Habitability Scoring & Ranking

The final model's predict_proba() method returns a probability score between 0 and 1 for each planet. This probability becomes the habitability score used to rank candidates:

Score Range	Label	Meaning
0.75 – 1.00	Habitable	Strong indicators of life-supporting conditions

0.50 – 0.74	Potentially Habitable	Some promising indicators, further study needed
0.25 – 0.49	Unlikely	Marginal conditions, extreme caution advised
0.00 – 0.24	Non-Habitable	Extreme conditions incompatible with known life

The ranked output is saved as `data/processed/habitability_ranked.csv` with all planets ordered from most to least habitable.

3.9 Model Interpretability

Feature importance is extracted from the Random Forest to explain which planetary and stellar parameters most influence the habitability prediction. This is visualised as a horizontal bar chart and documented with scientific reasoning. Typical top features include equilibrium temperature, orbital semi-major axis, stellar temperature, and planet radius.

Model Deliverables:

- `models/random_forest.pkl` — serialised with joblib
- `models/xgboost.pkl` — serialised with joblib
- `data/processed/habitability_ranked.csv`
- `notebooks/` — Jupyter notebooks with training code and plots

4. Module 3 — Flask Backend API

The trained machine learning model is exposed through a Flask-based REST API. Flask receives HTTP requests from the React frontend, validates the input, runs it through the loaded ML model, and returns a structured JSON response containing the habitability prediction and score.

4.1 Backend Environment & Dependencies

The backend runs inside a Python virtual environment. All dependencies are listed in requirements.txt for reproducible installs:

```
Flask joblib numpy pandas scikit-learn xgboost flask-cors
```

4.2 Backend Folder Structure

```
backend/
├── app.py # Main Flask application – all routes
├── utils.py # Helper functions (preprocessing, scoring)
├── models/
│   ├── random_forest.pkl
│   ├── xgboost.pkl
│   └── data/
│       └── exoplanets.csv
└── requirements.txt
```

4.3 Loading the ML Model

The trained model is loaded once when the Flask app starts — not on every request. This keeps prediction latency minimal:

```
import joblib from flask import Flask, request, jsonify from flask_cors import CORS app = Flask(__name__) CORS(app, origins=['https://exohabitai-2-0eyo.onrender.com']) # Load model at startup model = joblib.load('models/random_forest.pkl')
```

4.4 API Endpoints — Detailed Design

Endpoint 1: POST /predict

Method	POST
URL	https://exohabitai-1-3xkv.onrender.com/predict
Content-Type	application/json
Description	Accepts a single exoplanet's parameters and returns its habitability prediction.

Request Body:

```
{ "pl_rade": 1.2, "pl_bmasse": 1.5, "pl_orbper": 365.0, "pl_orbsmax": 1.0, "pl_eqt": 288, "st_teff": 5778, "st_lum": 1.0, "st_met": 0.0 }
```

Response:

```
{ "habitability_score": 0.82, "label": "Potentially Habitable", "confidence": "High", "status": "success" }
```

Endpoint 2: GET /rank or POST /rank

GET /rank	Returns top-N pre-ranked exoplanets from built-in dataset. Query param: ?top=N (default 10).
POST /rank	Accepts a JSON array of exoplanets, scores all, returns sorted by habitability score (desc).

Response	JSON array of planet objects each containing name, score, label, and input parameters.
----------	--

4.5 Input Validation

Every incoming JSON body is validated before the model is called. Missing or invalid parameters return a clear 400 error:

```
REQUIRED_FIELDS = ['pl_rade', 'pl_bmasse', 'pl_orbper', 'pl_orbsmax', 'pl_eqt', 'st_teff']
missing = [f for f in REQUIRED_FIELDS if f not in data]
if missing: return jsonify({'error': f'Missing fields: {missing}', 'status': 'error'}), 400
```

4.6 Response Structure

All responses follow a consistent JSON envelope:

```
# Success { "status": "success", "habitability_score": 0.82, "label": "...", ... } # Error
{ "status": "error", "error": "Missing fields: [pl_rade]", "code": 400 }
```

4.7 CORS Configuration

Flask-CORS is configured to allow requests specifically from the deployed React frontend domain, preventing unauthorised cross-origin access:

```
CORS(app, resources={ r"/predict": {"origins": "https://exohabitai-2-0eyo.onrender.com"},
r"/rank": {"origins": "*" } })
```

4.8 Testing the Backend

The API can be tested using:

- Browser — open GET /rank in the browser to verify the server is running
- Postman — send structured POST requests with JSON bodies
- curl — command-line testing example below

```
curl -X POST https://exohabitai-1-3xkv.onrender.com/predict \ -H "Content-Type: application/json" \
-d '{"pl_rade":1.2,"pl_bmasse":1.5,"pl_orbper":365,\
"pl_orbsmax":1.0,"pl_eqt":288,"st_teff":5778}'
```

5. Module 4 — React Frontend

The frontend is a single-page React application titled ExoHabitAI — Mission Control. It provides an intuitive space-themed UI where users can submit planet parameters, upload CSV files for batch predictions, and view ranked habitability leaderboards. The app communicates entirely with the Flask backend via HTTP requests.

5.1 Frontend Tech Stack

Framework	React 18
Build Tool	Vite — ultra-fast development server and bundler
Styling	Tailwind CSS with custom space / dark-mode theme
HTTP Requests	fetch API / axios — all calls to Flask backend
Routing	React Router v6 — multi-page SPA navigation
State Management	React useState and useEffect hooks
Theme	Space / Mission Control aesthetic — dark background, cyan and green accents
Environment Var	VITE_API_BASE_URL set to the backend Render URL

5.2 Pages & Components

The application contains the following pages:

- Home / Dashboard — welcome page, quick-start guide, live API status badge
- Single Planet Predictor — form with all input fields, submits to POST /predict
- Batch Upload — CSV file picker, submits to POST /predict-batch, renders results table
- Planet Ranker — paste multiple planet objects, calls POST /rank, displays leaderboard
- Top Exoplanets — calls GET /rank to list the best candidates from the built-in dataset

5.3 API Integration Example

```
const API = import.meta.env.VITE_API_BASE_URL; const handlePredict = async (formData) => {
  setLoading(true); try { const res = await fetch(`${API}/predict`, { method: 'POST',
    headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(formData) }); const
    data = await res.json(); setResult(data); } catch (err) { setError('Server is waking up...
    please retry in 30s'); } finally { setLoading(false); } };
```

5.4 Frontend Folder Structure

```
exohabitai-frontend/ ■■■■ src/ ■ ■■■■ App.jsx # Root component + routing ■ ■■■■ pages/ ■ ■
■■■■ Home.jsx ■ ■ ■■■■ Predict.jsx # Single-planet prediction form ■ ■ ■■■■ Batch.jsx # CSV
batch upload ■ ■ ■■■■ Rank.jsx # Planet ranker ■ ■ ■■■■ TopPlanets.jsx ■ ■■■■ components/
■ ■ ■■■■ ScoreGauge.jsx # Radial habitability indicator ■ ■ ■■■■ ResultCard.jsx # Score +
label display ■ ■ ■■■■ Navbar.jsx ■ ■ ■■■■ LoadingSpinner.jsx ■ ■■■■ api/ ■ ■■■■
exohabit.js # All fetch() wrappers ■■■■ .env # VITE_API_BASE_URL= ■■■■ vite.config.js ■■■■
tailwind.config.js ■■■■ package.json
```

6. Module 5 — Deployment on Render

Both services are deployed on Render.com, a modern cloud platform with direct GitHub integration. Every push to the main branch triggers an automatic redeploy of both services. Render issues HTTPS certificates automatically — no manual SSL configuration required.

6.1 Backend — Render Web Service

Service Type	Web Service (Python environment)
Live URL	https://exohabitai-1-3xkv.onrender.com
Runtime	Python 3.11
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>python app.py</code> or <code>gunicorn app:app --bind 0.0.0.0:\$PORT</code>
Port	Dynamically assigned by Render via <code>\$PORT</code> environment variable
Auto-Deploy	Yes — on every push to main branch
Cold Start	~30 seconds after 15 minutes of inactivity (free tier)
CORS Allowed From	https://exohabitai-2-0eyo.onrender.com

6.2 Frontend — Render Static Site

Service Type	Static Site (CDN-distributed)
Live URL	https://exohabitai-2-0eyo.onrender.com
Build Command	<code>npm install && npm run build</code>
Publish Dir	<code>dist/</code>
Framework	Vite (React)
CDN	Render's global CDN — assets served from edge nodes worldwide
Auto-Deploy	Yes — on every push to main branch
Sleep Behaviour	Static sites never sleep — always available instantly
Key Env Var	<code>VITE_API_BASE_URL = https://exohabitai-1-3xkv.onrender.com</code>

6.3 Deployment Process — Step by Step

1. Commit and push code to the GitHub repository (main branch).
2. Render detects the push via a GitHub webhook.
3. Backend: Render runs `pip install -r requirements.txt`.
4. Backend: Flask app starts; joblib loads the `.pkl` model into memory.
5. Frontend: Render runs `npm install` then `npm run build`, producing `dist/`.
6. Frontend: The `dist/` directory is pushed to Render's global CDN.
7. Both services go live with auto-issued HTTPS certificates.

6.4 Environment Variables on Render

Variable	Service	Value
VITE_API_BASE_URL	Frontend	https://exohabitai-1-3xkv.onrender.com
PORT	Backend	Auto-assigned by Render
PYTHON_VERSION	Backend	3.11 (via runtime.txt)

7. API Reference — All Endpoints

The complete interactive API documentation (Swagger UI) is available at: <https://exohabitai-1-3xkv.onrender.com/docs>

Method	Endpoint	Input	Output
GET	/	—	API status, version, model_loaded flag, endpoint index
GET	/health	—	Health check — server running + model loaded status
GET	/rank?top=N	Query param: top (int)	Top-N pre-ranked exoplanets from built-in dataset
POST	/predict	JSON — single planet	habitability_score, label, confidence, status
POST	/predict-batch	JSON array or CSV file	Array of predictions for every input row
POST	/rank	JSON array of planets	Sorted list by habitability score (descending)
POST	/train-and-predict	CSV/JSON raw dataset	Processes dataset and returns predictions for all rows

Try the live backend root: <https://exohabitai-1-3xkv.onrender.com>

8. End-to-End Data Flow

The complete journey of a single prediction from user click to screen:

```
1. USER fills the prediction form on the React frontend | | POST /predict (HTTPS JSON) v 2.
FLASK receives and parses the request body | v 3. INPUT VALIDATION – checks all required
fields are present | v 4. pandas DataFrame row is created from the input values | v 5.
scikit-learn Pipeline applies scaling to the row | v 6. Random Forest .predict_proba(row)
is called | v 7. Probability score (0.0 - 1.0) extracted from output | v 8. Threshold logic
assigns a human-readable LABEL | v 9. JSON response returned: {score, label, confidence} |
v 10. React setState() triggers re-render | v 11. ScoreGauge + ResultCard displayed to the
USER
```


9. Challenges & Solutions

9.1 Cold Start Latency (Free Tier)

Challenge: Render's free tier spins down the backend after 15 minutes of inactivity. The first request triggers a cold start taking 20-40 seconds while Python restarts and joblib reloads the model.

Solution: The React frontend shows a loading spinner with a 'Waking up the server...' message. A future improvement is to use UptimeRobot to ping GET /health every 14 minutes.

9.2 CORS Errors

Challenge: Browsers block cross-origin requests by default. The React frontend (on one Render subdomain) calling the Flask backend (on a different subdomain) causes CORS preflight failures.

Solution: flask-cors is installed and CORSMiddleware is configured with the exact frontend Render URL as the allowed origin.

9.3 Class Imbalance in Training Data

Challenge: The exoplanet dataset is heavily imbalanced — far more non-habitable planets exist than habitable ones. A naive model would always predict non-habitable and still achieve high accuracy.

Solution: `class_weight='balanced'` is passed to `RandomForestClassifier`. Recall is prioritised over accuracy in model selection. SMOTE (Synthetic Minority Oversampling) is optionally applied for the XGBoost model.

9.4 Missing Values in Production Inputs

Challenge: Real exoplanet databases often have incomplete records. A missing column in a prediction request would crash the model.

Solution: The API validates all required fields before calling the model. Non-critical missing fields are imputed with training-set medians stored alongside the serialised model.

9.5 Large Batch CSV Files

Challenge: Large CSV uploads can time out or consume excessive memory on a free-tier server with 512MB RAM.

Solution: The frontend validates file size before upload. The backend uses pandas chunking for very large files. `Model.predict_proba()` is vectorised so entire DataFrames are scored in a single call — no Python loops.

10. Complete Project File Structure

Backend Repository

```
exohabitai-backend/ ■■■ app.py # Flask app – all routes ■■■ utils.py # Preprocessing +
scoring helpers ■■■ models/ ■ ■■■ random_forest.pkl # Trained model (joblib) ■ ■■■
xgboost.pkl ■■■ data/ ■ ■■■ raw/ # Original NASA dataset ■ ■■■ preprocessed/ ■ ■ ■■■
preprocessed.csv # Cleaned dataset ■ ■■■ processed/ ■ ■■■ exohabit_ml.csv ■ ■■■
habitability_ranked.csv ■■■ notebooks/ ■ ■■■ 01_preprocessing.ipynb ■ ■■■
02_model_training.ipynb ■■■ requirements.txt ■■■ runtime.txt # python-3.11.x ■■■
render.yaml # Render deploy config
```

Frontend Repository

```
exohabitai-frontend/ ■■■ src/ ■ ■■■ App.jsx ■ ■■■ pages/ ■ ■ ■■■ Home.jsx ■ ■ ■■■
Predict.jsx ■ ■ ■■■ Batch.jsx ■ ■ ■■■ Rank.jsx ■ ■ ■■■ TopPlanets.jsx ■ ■■■
components/ ■ ■ ■■■ ScoreGauge.jsx ■ ■ ■■■ ResultCard.jsx ■ ■ ■■■ Navbar.jsx ■ ■ ■■■
LoadingSpinner.jsx ■ ■■■ api/ ■ ■■■ exohabit.js ■■■ .env # VITE_API_BASE_URL ■■■
vite.config.js ■■■ tailwind.config.js ■■■ package.json
```

11. How to Run Locally

Backend

```
# 1. Clone the backend repository git clone cd exohabitai-backend # 2. Create and activate
virtual environment python -m venv venv source venv/bin/activate # Windows:
venv\Scripts\activate # 3. Install dependencies pip install -r requirements.txt # 4.
(Optional) Retrain the model python notebooks/train.py # 5. Start Flask dev server python
app.py # API available at: http://localhost:5000 # Swagger docs at:
http://localhost:5000/docs
```

Frontend

```
# 1. Clone the frontend repository git clone cd exohabitai-frontend # 2. Install Node
dependencies npm install # 3. Point to local backend echo
'VITE_API_BASE_URL=http://localhost:5000' > .env # 4. Start Vite dev server npm run dev #
Frontend available at: http://localhost:5173
```

12. Summary & Live Links

ExoHabitAI is a complete end-to-end AI product that demonstrates the full data science lifecycle: from raw planetary data to a deployed, publicly accessible prediction platform. The project integrates data engineering, supervised machine learning, REST API development, and modern frontend development — all within a space exploration theme.

What We Built:

- Data Preprocessing pipeline — cleaned, engineered, and standardised NASA exoplanet data
- ML Model — Random Forest (+ XGBoost comparison) with hyperparameter tuning
- Flask REST API — 7 endpoints exposing predictions, ranking, and batch processing
- React Frontend — space-themed SPA with forms, CSV upload, and ranked leaderboards
- Cloud Deployment — two services live on Render.com with HTTPS and auto-deploy

How We Did It:

1. Collected raw data from NASA Exoplanet Archive
2. Preprocessed: imputed missing values, removed outliers, standardised units, engineered features
3. Trained Random Forest and XGBoost classifiers; tuned with GridSearchCV
4. Serialised the best model with joblib and built a Flask API around it
5. Built the React frontend with Vite and connected it to the Flask backend
6. Pushed both services to GitHub; Render auto-deployed them on every commit

LIVE DEPLOYMENT — CLICK TO OPEN

Frontend App: <https://exohabitai-2-0eyo.onrender.com>

Backend API: <https://exohabitai-1-3xkv.onrender.com>

API Docs (Swagger): <https://exohabitai-1-3xkv.onrender.com/docs>